

State management is the way we store, update, and share data (state) in an application so that the UI stays in sync with the data.

Software Development Life Cycle (SDLC) in simple words:

It is a step-by-step process used to create software from start to end.

1. Requirement Gathering – What do we need to make?
2. Planning – How to make it and what tools to use?
3. Design – Structure, UI, database layout.
4. Development – Actual coding part.
5. Testing – Check if everything works properly or has bugs.
6. Deployment – Release the software to users.
7. Maintenance – Fix issues, add updates after release.

React Router is a library used in React to handle navigation between different pages without reloading the browser.

Diff in find , findone , findbyid

https://chatgpt.com/s/t_6982ecbc1cf0819191ad11873aaf5c59

In Express, `express.static` is used to serve static files like images, CSS, and JavaScript.

`app.use(express.static('Public'))` allows the browser to directly access all files inside the `Public` folder.

`app.use('/Images', express.static('Public/Images'))` creates a URL path `/Images` that maps to the `Public/Images` folder, which is commonly used to serve uploaded images.

This helps the frontend access files directly using a URL without writing backend routes.

localStorage

localStorage stores data permanently in the browser.

Key points

- Data does not expire
- Data remains even after browser restart
- Shared across all tabs of the same origin
- Maximum size $\approx$ 5–10 MB
- Stored as string only

We manually clear data using `.clear()`

sessionStorage

sessionStorage stores data only for one browser tab session.

Key points

- Data is cleared when tab or window is closed
- Not shared between tabs
- Maximum size $\approx$ 5 MB
- Stored as string only

Session means the server stores user-related information after login, so it knows the user is authenticated on every request. After login, the server gives the client a session ID, which is usually stored in a cookie. When the client sends a request, it (client) sends this ID. The server uses that ID to find the user's info it saved earlier. **The server checks whether the session ID sent by the client exists in the server's session store.**

Session is slightly slower.

Session is used when we need short term authentication.

Serverless architecture means you don't manage servers manually
The cloud runs your code only when needed, and you pay only for the time it runs.

Third-party cookies are cookies made by a different website, not the one you are visiting.

Third-party cookies are cookies created by another website to track users and show ads.

A **gateway** is a device or software that connects two different networks and allows them to communicate with each other.

HTTP (HyperText Transfer Protocol) is a set of rules used to transfer data between a web browser and a web server.

Rule : stateless , client - server , use https methods , use proper status code.

Dynamic typed language means you don't need to declare the data type of a variable — the type is decided automatically at runtime.

Indexing in MongoDB is used to speed up data searching.

Without an index, MongoDB checks each document one by one (full collection scan).

With an index, MongoDB can directly jump to the required data, just like a book index helps you find a topic quickly.

Stream means data flowing in small parts (chunks) instead of all at once.

Buffer is like a temporary storage where Node.js stores raw binary data before using it.

Think like:

You download a file — data doesn't come all at once, it comes in small parts.

These parts are first stored in Buffer, then processed/outputted.

A **server** is a powerful computer or system that provides services, data, or resources to other computers (called *clients*) over a network.

When we log in, the server automatically stores a JWT token inside a cookie. Later, this cookie is sent with every request, so we can access protected routes. But if we don't use `withCredentials: true`, the cookie won't be sent along with the request, and without the JWT token, we cannot access protected routes.

Zod ->

https://chatgpt.com/s/t_692dc66ece8481919b5f402eeae365d1

Promise chaining means running tasks one after another where each next task starts only after the previous one finishes.

We connect them using `.then()`.

setImmediate()

It runs as soon as possible, but not immediately.

It waits for Node.js to finish the current work.

Fork is a method in NodeJS that is used to create child processes.

app.use(express.json());

Body-parser is a tool in Node.js/Express that helps us read the data sent by the client in the request body.

What is the purpose of NODE_ENV? (Simple Words)

NODE_ENV tells your Node.js app which mode it is running in:

development (for coding/testing)

production (for deployment)

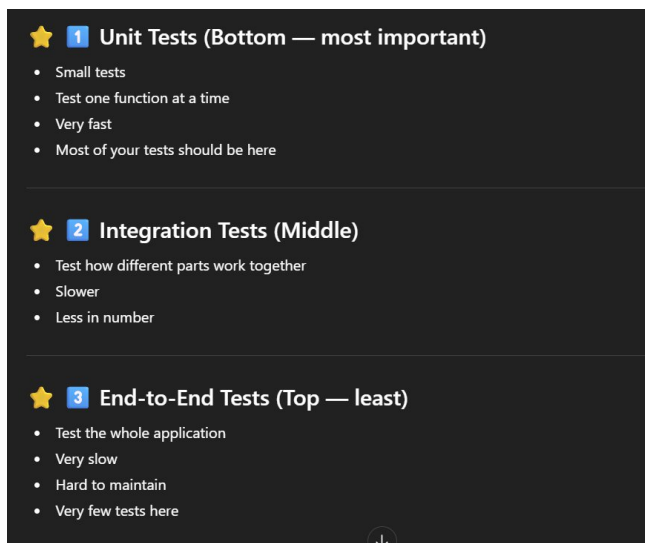
test (for testing tools)

We use for development , production is only for https

It helps your app behave differently in each mode.

The **Test Pyramid** is a simple idea for software testing that says:

Most tests should be at the lower levels, and the number of tests should decrease as you go upward.



The **Redis module** in Node.js is a package that allows your Node.js app to connect to Redis, store data, and get data from Redis.

The **Util module** in NodeJS provides access to various utility functions. There are various utility modules available in the NodeJS module library.

Utility functions are small helper functions that perform simple, common tasks and can be reused in many places in a program.

Os module,path module,DNS module

Nodemailer Working :

You create a transporter

What is a Transporter?

A transporter is an object that contains the configuration and methods needed to send emails through a specific email service provider (like Gmail, Outlook, etc.).

This is like connecting to an email service (Gmail, Outlook, SMTP server, etc.)

You prepare the email details
(to, subject, text, HTML, attachments)

Nodemailer sends the email using SMTP

SMTP = protocol used to send emails.

Email goes through the mail server

(Gmail or your custom email server)

Recipient receives the email

It appears in the inbox.

```
class EmailService {
  constructor() {
    this.transporter = nodemailer.createTransport({
      service: "gmail",
      secure: true,
      port: 465,
      auth: {
        user: "himanshudhage123@gmail.com",
        pass: "csuq pznw vaks vkpp"
      }
    });
  }
}
```

Npm init -y => is used to initialize new Node.js project , -y is used to skip all questions and automatically fills default values. Like package name, version

Npx -> it allows us to run node.js package , without installing it globally on our system.

Nodemon => it automatically restart server whenever our code changes.

NodeJS is a runtime environment for executing JavaScript outside the browser, built on the V8 JavaScript engine

A **web-hook** is a way for a server to send data automatically to another server when a specific event occurs.

Without sending request server send response when specific event happen/occurs

Usually, we can't run JavaScript outside the browser because it needs a JavaScript engine to execute code, and that engine is provided by the browser.

Node is cross platform so it can run on any platform.

Event Driven means operation in node JavaScript are based on events.

Module : ideally JavaScript file is treated as module.

Encryption is the process of converting readable data (plain text) into unreadable form (cipher text) so that unauthorized people cannot read it.

Decryption is the reverse process of encryption — it converts cipher text back into readable plain text.

Encryption and decryption

Web apis

Routing --> Routing in Node.js (or Express.js) is used to direct incoming HTTP requests to the appropriate methods based on the URL and HTTP

Nodemailer :

Types of NoSQL Database

1. Document-Based:

- Stores data as documents (usually JSON, BSON or XML format).
- Each document can have different flexible fields.
- ✓ **Popular Databases:** MongoDB, CouchDB
- ✓ **Used In:** Websites, Blogs, Content Management



2. Key Value Pair:

- It uses a hash table that stores a pointer, which is the key, which then points to a value that stores some form of information or data.
- Handle lots of data and heavy load.
- ✓ **Popular Databases:** Redis, DynamoDB
- ✓ **Used In:** Caching, User sessions, Real-time apps

```
{
  "image": "Cheryl-Carter.jpg",
  "firstname": "Cheryl",
  "lastname": "Carter",
  "company": "Skyble",
  "email": "ccarter@gmail.com",
  "phone": "2-(017)772-7449",
}
```

method

Types of NoSQL Database

3. Column-Based:

- Stores data in columns instead of rows.
- Good for reading large volumes of data quickly.
- ✓ **Popular Databases:** Apache Cassandra, Hbase
- ✓ **Used In:** Analytics, IoT, log data

partition key	email	name	tel	columns ...
101	ab@c.to	otto	12345	
103	karl@a.b	karl	6789	tel2
104		linda		

4. Graph-Based:

- Stores data as nodes (items) and edges (relationships).
- Best for understanding relationships between data.
- ✓ **Popular Databases:** Neo4j, Amazon Neptune
- ✓ **Used In:** Social Networks, Fraud detection, Recommendation engines



→ do not need to define structure. Advance use stored diff kinds of data together.

Page No.	
Date	

db scaling

① cache ~~not~~ ready connection fact still

Adv

→ Our All relevant data is contain together makes insertion and deletion faster.

Adv

retrieve.

- schema is easily changable, in relational db. Adding new Attribute is expensive
- Built for scale.
- Built for aggregation and analysis.
- High speed & real-time processing (chat, streaming, gaming)

dis-Adv.

- does not follow ACID / ACID is not gurantee.
- That way financial system does not use the NOSQL for transaction because it doesn't make more sense
- Reading is often slower than SQL because is less structured
- In NOSQL, db, relationship are not implicit, means: table/collections don't automatically know are they are related. - you have to handle it

key-value pair: provide high speed read & write data. eg. Redis

- whatsapp.

columnwise - (Horizontal Scaling). Easily handle more user or data adding more server.

Cassandra, Netflix to manage million of user watching videos at a time.

SQL → Banking system, E-commerce (Social media App) for order user payment.

👉 Use NoSQL when:

- You need **schema flexibility** (frequent changes in data model).
- You want **faster insertion, deletion, and retrieval** of data.
- The system must be **scalable** to handle large amounts of data or users.
- You need **real-time, high-speed processing** (e.g., chat apps, streaming, gaming).
- Use cases like:
 - **Redis** → high-speed read/write (WhatsApp).
 - **Cassandra** → managing millions of concurrent users (Netflix).

👉 Do not use NoSQL when:

- You need **strict ACID transactions** (e.g., banking, financial systems).
- Relationships between data are important and must be **implicit** (SQL handles this better).

✅ When to Use SQL

1. ACID Transactions are Required

- Strong data consistency, atomic operations, rollback on failure.
- Example: **Banking systems, payment processing.**

2. Data is Highly Structured

- Well-defined schema (tables, rows, columns).
- Relationships between data are important (foreign keys, joins).

3. Complex Queries & Reporting

- SQL supports joins, aggregations, subqueries → ideal for analytics and reporting.

The cloud refers to delivering computing services like servers, storage, databases, networking, and software over the internet instead of running them on local machines.

latest stable version of React is 19.1.0

the latest stable version of Node.js is v24.9.0

Vite is rapid development tool for modern web projects. focus on speed and performance by improving the development experience.

CommonJS is a module system used in Node.js to organize and reuse code.

It allows you to split your code into separate files and use them wherever you need.

Uses `require()` to import modules and `module.exports` to export them.

Synchronous – Code runs line by line.

Mainly used in Node.js (before ES6 modules).

ECMAScript (ES) is the standard specification for JavaScript.

JavaScript is a programming language, and ECMAScript defines its rules, syntax, and features.

Different versions of ECMAScript add new features to JavaScript, like `let/const`, arrow functions, classes, modules, `async/await`, etc

ECMAScript 2015

“The latest ECMAScript version is ES2024, also called ES15.”

ES6 introduced let/const, arrow functions, template literals, classes, destructuring, spread/rest operators, promises, modules, and new data structures like Map and Set..

Is js single threaded / node js is single threaded

Promises , async await js video

Private key cryptography

Private key cryptography uses one single key for both encryption and decryption.

The same secret key is shared between sender and receiver.

The sender encrypts the message using the key.

The receiver decrypts the message using the same key.

Public Key

The public key is mainly used for encrypting data

We can distribute public key to anyone else and encrypted data is decrypted by our private key.

Why do we use JWT

it is used for authorization, authentication, to exchange Information Client Server

Encoding :

Encoding is used to convert data into another format so it can be safely transmitted or stored, but anyone can decode it back.

Base64 encoding

It's called Base64 because it uses 64 different characters to represent data.

Working :

1. In the computer, all data is stored as binary (0s and 1s).
2. Base64 takes the binary data and divides it into groups of 6 bits (because $2^6 = 64$ — that's why it's called Base64).
3. Each 6-bit group is then mapped to a specific character (A-Z, a-z, 0-9, +, /).
4. If the total bits don't divide evenly by 6, it adds padding (=) at the end.

What is the purpose of the header in a JWT?

The header contains metadata about the token —

basically information that helps the server understand how to verify it.

What are the security implications(Risks + Precautions) of using JWT?

If someone steals the JWT (like from localStorage), they can act as that user until it expires.

JWT payload is only *encoded*, not encrypted — anyone can read it if they get the token

In JWT

JWT Header & Payload are encoded (Base64URL) — anyone can decode them.

JWT Signature is signed — only the server with the secret key can verify it.

app.use(fileUpload(...))

This activates the file upload feature for all routes in your app.

Without this, Express can't understand files sent from the front end.

And also store it on temporary folder

In symmetric encryption, the same key is used for both encrypting and decrypting so anyone with that key can decrypt.

In asymmetric encryption, there are two keys — public and private.

Data encrypted with the public key can only be decrypted with the private key

Diff between httponly and http cookies

Normal Cookie (http cookie)

A regular cookie can be read and written using JavaScript (document.cookie).
This means client-side scripts can access it.

HttpOnly Cookie

Special flag added while setting the cookie (HttpOnly).
Such cookies cannot be accessed by JavaScript (document.cookie won't show them)

```
res.clearCookie("jwt");
```

Is use to clear cookie

How sever verifies the token

1. Secret key फक्त server कडे असते, token मध्ये नाही.
2. Token मध्ये फक्त Header + Payload + Signature असतो.
 - Signature हा Header + Payload + secret key वापरून server ने बनवलेला hash आहे.
3. जेव्हा server ला token मिळतो:
 - Server आपल्या secret key वापरून Header + Payload वरून Signature पुन्हा generate करतो
 - नंतर तो compare करतो received token च्या Signature सोबत
4. Match झाल्यास → token valid, नाही तर invalid

"Signature in JWT is used to verify the authenticity and integrity of the token.

When the server receives a token, it uses its secret key to generate a new signature from the received header and payload, and then compares it with the signature part of the received token.

If both match, the token is valid; otherwise, it is considered tampered or invalid."

JWT is StateLess

Stateless means:

The server does not remember anything about the user between requests.

Jwt Works on Private Key Cryptography

What is JWT (JSON Web Token)?

JWT stands for JSON Web Token.

It is a secure way to share information between two parties — like between a client (browser) and a server (backend).

What is HS256 Algorithm in JWT?

Hash-based Message Authentication Code

HMAC (Hash-Based Message Authentication Code)

HMAC is a way to verify both the integrity and authenticity of a message using a cryptographic hash function and a secret key.

Key points:

1. **Integrity:** Ensures the message hasn't been modified.
2. **Authenticity:** Ensures the message comes from someone who knows the secret key.

decrypted->

Structure of JWT using HS256:

<Header>.<Payload>.<Signature>

Header:

```
{  
  "alg": "HS256", "typ": "JWT"  
}
```

typ -> type of token

Payload:

contains the actual data (claims) you want to send — usually about the user or session.

```
{  
  "user": "himanshu",  
  "role": "admin",  
  "exp": 1718899200  
}
```

Signature

Secret key

A private string or value known only to the server (or trusted parties).

Used to sign the JWT (create the signature) and later verify it.

it is never sent to the client.

How JWT token Works?

1. **User Logs In:** The client (browser) sends login credentials to the server.
2. **Server Generates JWT:** If credentials are valid, the server creates a JWT containing user data and signs it with a secret key.
3. **Token Sent to Client:** The JWT is sent back to the client and stored (usually in localStorage or a cookie).
4. **Client Sends Token in Requests:** For protected routes, the client includes the JWT in the Authorization header (Bearer Token).
5. **Server Verifies and Responds:** The server verifies the token, extracts user info, and processes the request if valid.

A **Bearer Token** is a type of access token that is included in the Authorization header of an HTTP request to authenticate the user.

The **Authorization header** is an HTTP request header used to send credentials or tokens to a server so it can verify *who you are* and *what access you have*.

The header is the first part of a JWT and contains metadata about the token, like the type of token and the algorithm used to sign it.

Authorization means checking what a user is allowed to do after login.

Authentication is the process of verifying the identity of a user or system.

An **HTTP cookie** is a small piece of data that a server sends to a web browser, and the browser stores it and sends it back to the server with future requests.

The **HttpOnly** flag is a cookie attribute that tells the browser:

This cookie can only be sent in HTTP(S) requests, and JavaScript running in the browser cannot access it.

How to Securely Store JWT Tokens

1. HTTP-only Cookies (Recommended for security)

A cookie that JavaScript cannot access

But how it secure->

HTTP-only Flag

When a cookie has **HttpOnly=true**, JavaScript cannot read or write it.

This protects against XSS attacks,

XSS (Cross-Site Scripting)

XSS lets a hacker run their own JavaScript on your website without permission.

but try to keep short expiry time to secure token

Using the Access Token

Client sends the access token in every request (usually in the **Authorization: Bearer <token>** header).

Server checks the token:

If valid → allow request.

If expired → client must use the refresh token to get a new access token.

Access token expires

Either the server responds with **401 Unauthorized**

Or your frontend has a timer that knows the expiry time.

```
if (response.status === 401) {  
  await refreshAccessToken(); // automatic  
  retryOriginalRequest();  
}
```

Bcz we already written code in frontend, then it checks / reads 401 status and send refresh token req to the server and When a refresh request comes:

Server finds the refresh token in the database.

How can we invalidate Jwt token

1.by token expiry

A **refresh token** is like a backup key that lets you get a new access token when the old one expires — without logging in again.

```
const refreshToken = jwt.sign(payload, 'secret', { expiresIn: '30d' });
```

Access Token:

An access token is a small, secure code that proves you're allowed to use certain features or view protected data in an app or website.

What is a Session (in simple words)?

A session is a way for the server to remember a user across multiple requests — like saying: “You’re still logged in!”

How to Decode a JWT Without Verifying It

Split the token by dot(.)

Decode the header (first part)

Decode the payload (second part)

Ignore the signature (third part)

How do you handle token expiration in JWT?

Set an Expiration Time (exp)

What actually happens:

Frontend sends a request to the backend.

If JWT is expired, backend replies with 401 Unauthorized.

Frontend checks this response and logs the user out.

The token becomes invalid.

The next time you do anything (e.g., reload, click, navigate), the backend says: “You're not authorized!”

The frontend then:

Clears the token/cookie

Shows the login page

```
'secretKey', { expiresIn: '15m' });
```

This token will automatically expire after 15 minutes.

How can you refresh a JWT token?

✓ Step-by-Step:

Login → Server gives:

✓ Access Token (short-lived)